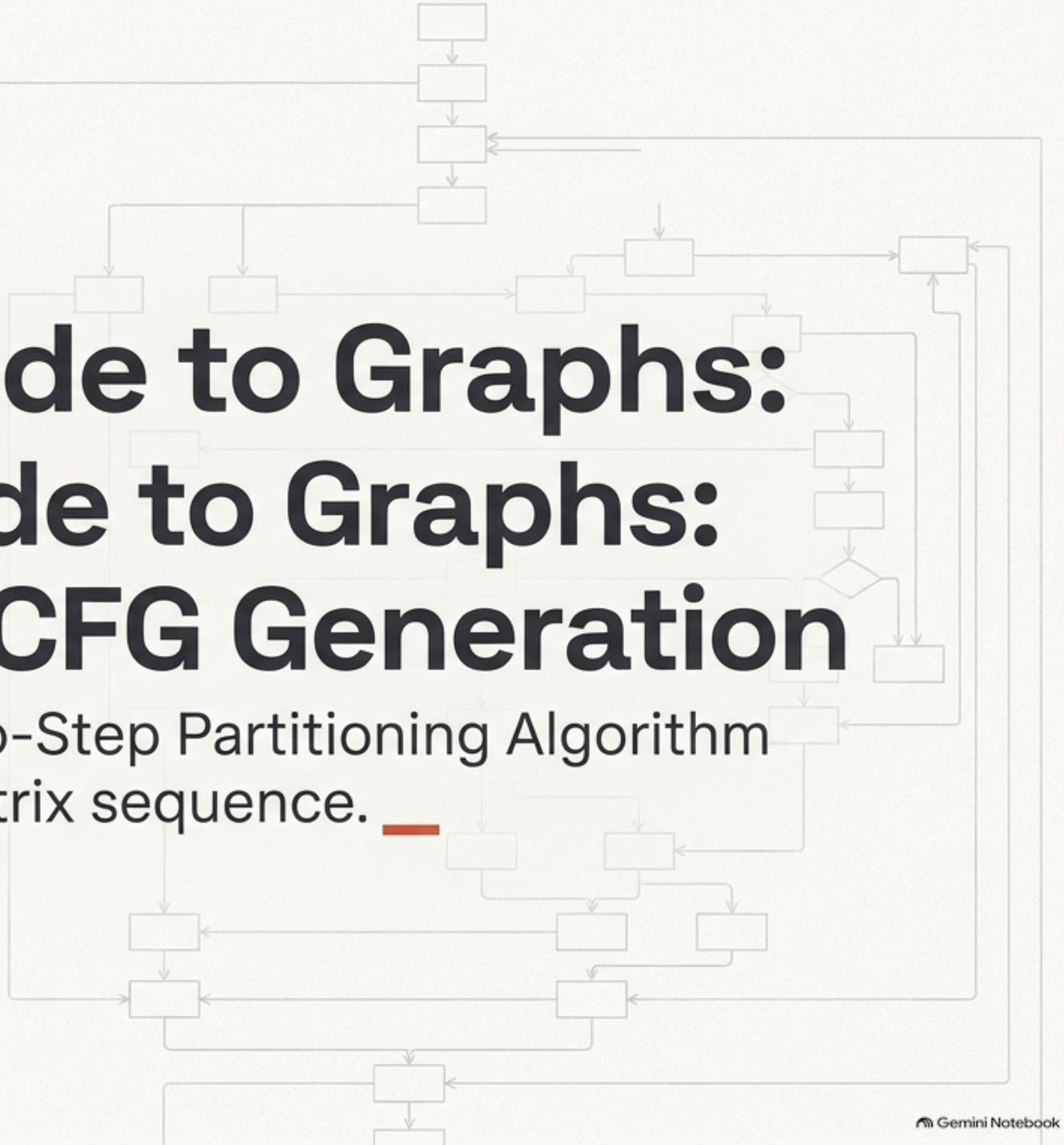
Translating Code to Graphs: Translating Code to Graphs: Basic Blocks & CFG Generation

A visual walkthrough of the Two-Step Partitioning Algorithm using the canonical 17-line matrix sequence.



Gemini Notebook

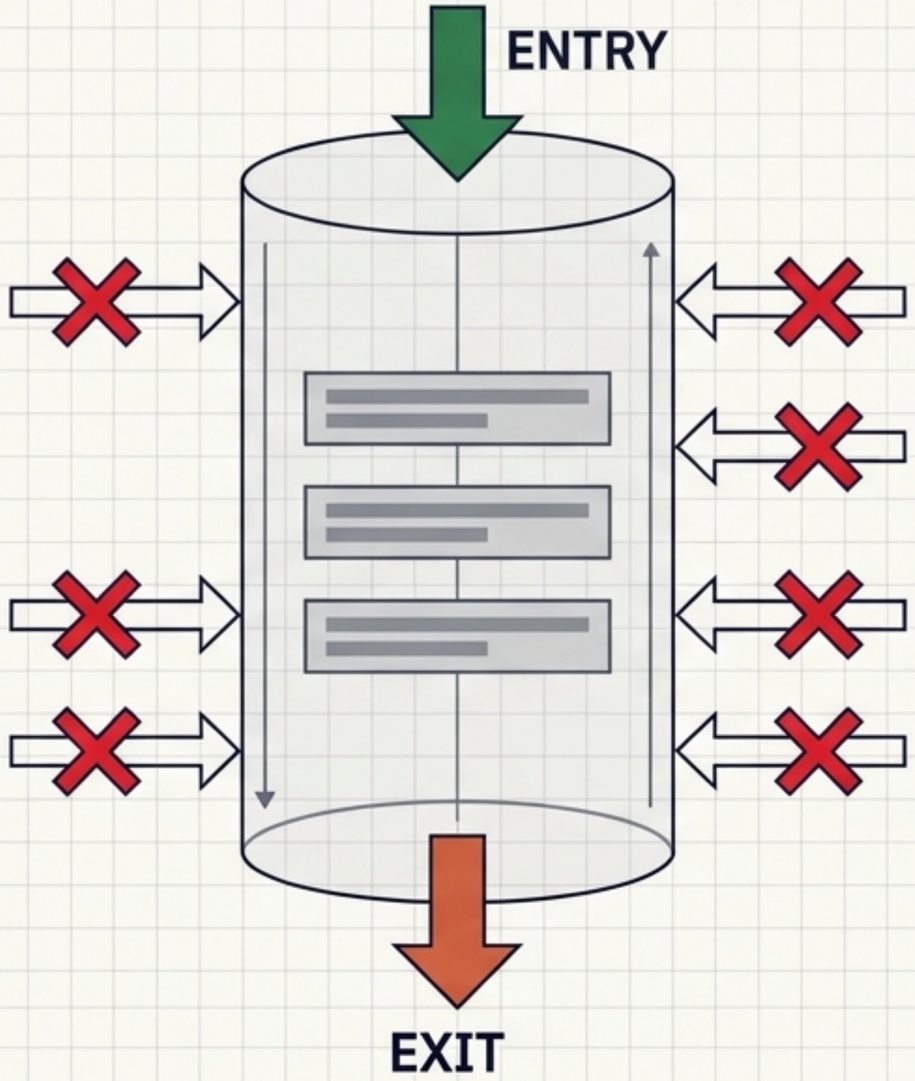
The Raw Material: Three-Address Code (3AC)

```
 1) 1) j = 1
 2) 3) t1 = 10 * i
 4) 4) t2 = t1 + j
 5) 5) t3 = 8 * t2
 6) 6) t4 = t3 - 88
 7) 7) a[t4] = 0.0
 8) 8) j = j + 1
 9) 9) if j <= 10 goto (3)
10) 10 i = i + 1
11) 11 if i <= 10 goto (2)
12) 12 i = 1
13) 13 t5 = i - 1
14) 14 t6 = 88 * t5
15) 15 a[t6] = 1.0
16) 16 i = i + 1
17) 17 if i <= 10 goto (13)
```

Context: This intermediate code converts a 10x10 matrix into an identity matrix. To a compiler, this flat sequence of instructions is difficult to analyze and optimize without exposing its underlying structural logic.

Gemini Notebook

Defining the Basic Block



A Basic Block is a maximal sequence of consecutive instructions with strict control flow properties.

- **Single Entry:** Control enters only through the first instruction. No jumping into the middle.
- **Single Exit:** Control leaves without halting or branching, except at the very last instruction.

ENGINEERING FOOTNOTE

The All-or-Nothing Guarantee: If the first instruction executes, every subsequent instruction in the block is guaranteed to execute exactly once.

Gemini Notebook

From Flat Code to Structured Blocks

Algorithm 8.5: The Two-Step Partitioning Process

Step 1: Identify Leaders



Determine the distinct entry points in the code. Every basic block must begin with a Leader.

Step 2: Delineate the Blocks



For each leader, its Basic Block consists of the leader itself, plus all subsequent instructions up to (but not including) the next leader.

Gemini Notebook

Step 1: The Three Rules of Leader Identification



Rule 1: The First Instruction

The very first line of the intermediate code sequence is automatically designated as a leader.



Rule 2: Jump Targets

Any instruction that is the destination of a **conditional** or **unconditional jump** (`goto`) is a leader.



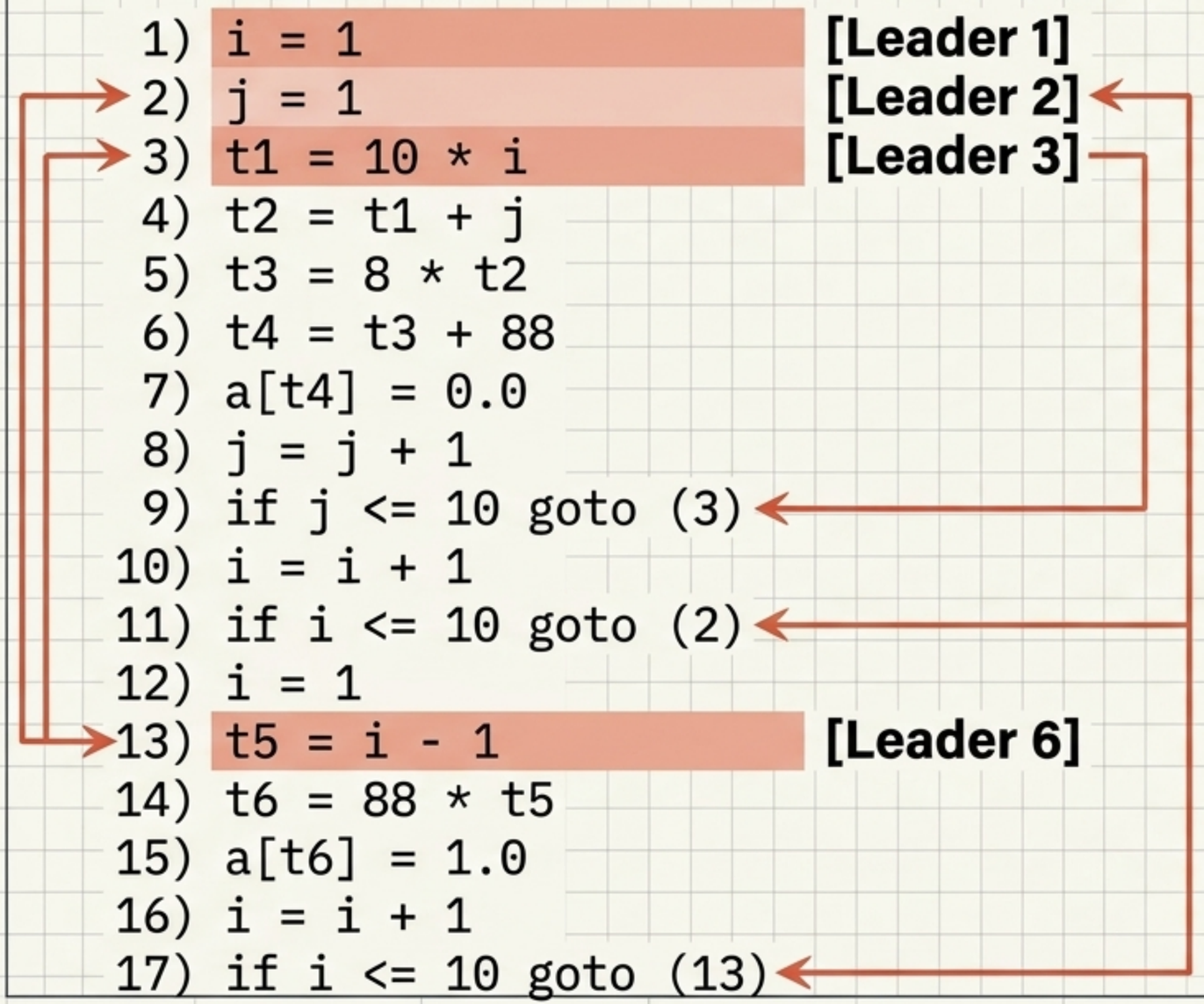
Rule 3: Jump Followers

Any instruction that immediately follows a **conditional** or **unconditional jump** is a leader.

Gemini Notebook

Finding Leaders: First Instruction & Jump Targets

```
 1) i = 1                          [Leader 1]
 2) j = 1                          [Leader 2]
 3) t1 = 10 * i                    [Leader 3]
 4) t2 = t1 + j
 5) t3 = 8 * t2
 6) t4 = t3 + 88
 7) a[t4] = 0.0
 8) j = j + 1
 9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1                     [Leader 6]
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```



Rule 1 Applied:
- Line 1 is flagged as Leader 1.

Rule 2 Applied:
- Line 9 targets Line 3 (Leader 3).
- Line 11 targets Line 2 (Leader 2).
- Line 17 targets Line 13 (Leader 6).

Gemini Notebook

Finding Leaders: Instructions Following Jumps

```
 1)   Previous leaders
 2)   previous leaders
 3)   previous leaders
 4)  goto = 0
 5)   i = i + 1
 6)   int i = 0
 7)   conote goto(1)
 8)   goto goto(3)
 9)
10)   i = i + 1                    [Leader 4]
11)
12)   i = 1                        [Leader 5]
13)   previous leaders
14)   i = i + 1
15)   i = i + 2
16)   print(i i, 0)
17)   goto(3)
```

Rule 3 Applied: We must flag instructions that execute if a conditional jump "falls through" (i.e., the condition evaluates to false).

- Line 10 immediately follows goto(3) (Leader 4).
- Line 12 immediately follows goto(2) (Leader 5).

Note: Line 17 is a jump, but it is the final line. There is no line 18, so no additional leader is formed here.

Gemini Notebook

Delineating the Basic Blocks (Step 2)

```
[B1   1) i = 1
[B2   2) j = 1
      3) t1 = 10 * i
      4) t2 = t1 + j
      5) t3 = 8 * t2
[B3   6) t4 = t3 + 88
      7) a[t4] = 0.0
      8) j = j + 1
      9) if j <= 10 goto (3)
[B4  10) i = i + 1
     11) if i <= 10 goto (2)
[B5  12) i = 1
     13) t5 = i - 1
     14) t6 = 88 * t5
[B6  15) a[t6] = 1.0
     16) i = i + 1
     17) if i <= 10 goto (13)
```

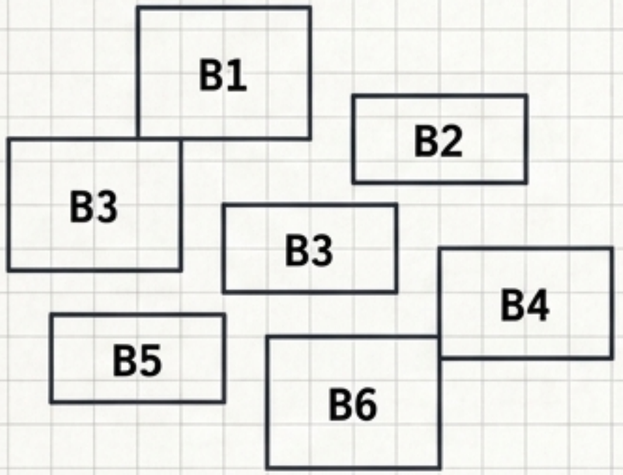
The sequence is now fully partitioned.

Rule: A block perfectly maps from a Leader down to the instruction immediately preceding the next Leader.

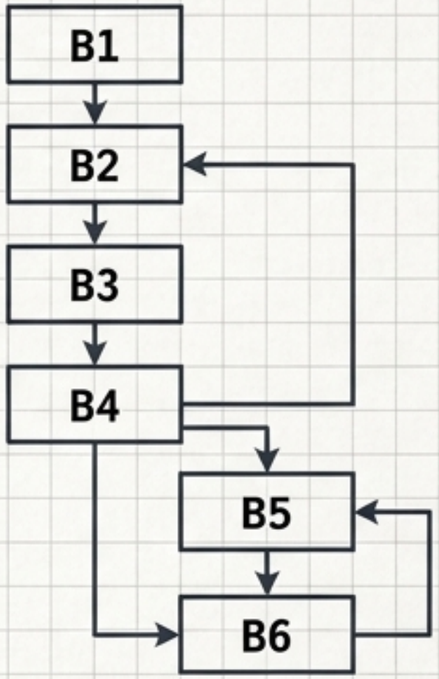
Gemini Notebook

Reconstituting Logic: The Control Flow Graph (CFG)

Fragmented Basic Blocks



Directed Flow Graph



Nodes: The Basic Blocks themselves. Each node guarantees straight-line execution.

Edges: Directed arrows indicating possible control transfers.
- **Jump Edge**: Block X ends in a jump targeting the leader of Block Y.
- **Fall-through Edge**: Block Y immediately follows Block X, and Block X does not end with an unconditional jump.

Gemini Notebook

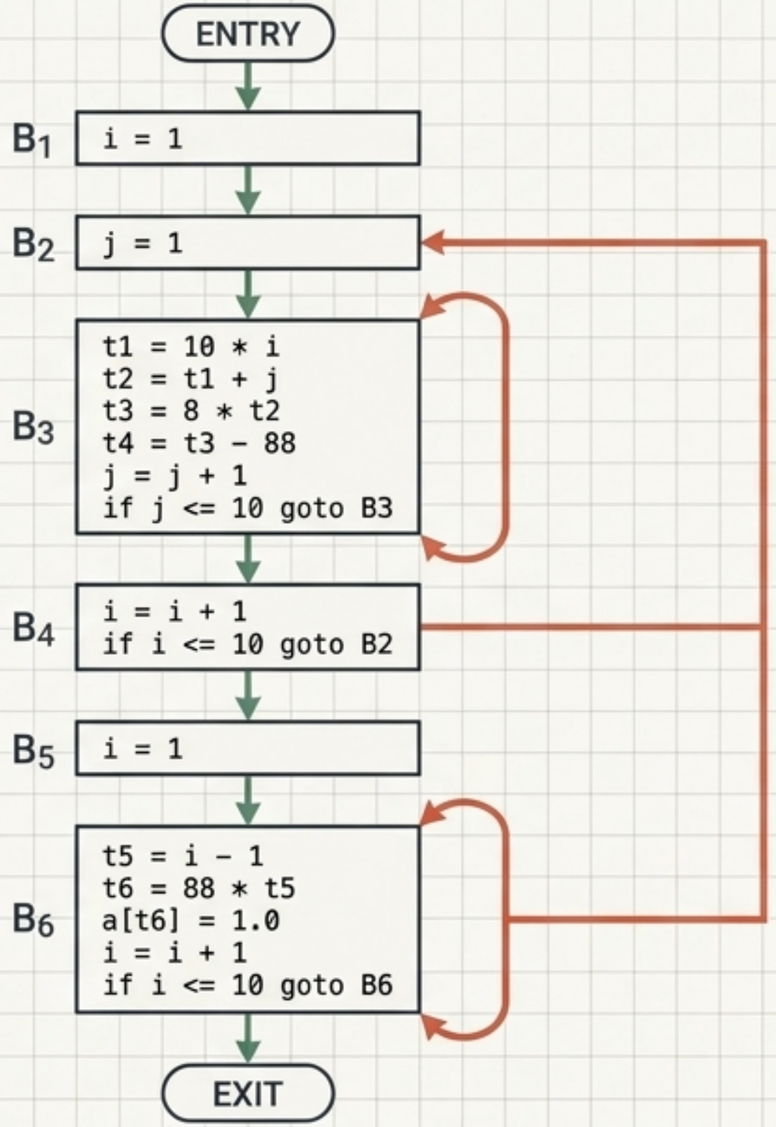
Determining Graph Connectivity

Source Node	Outbound Edges (Destination & Type)
B1	B2 (Fall-through)
B2	B3 (Fall-through)
B3	B3 (Jump to line 3) AND B4 (Fall-through to line 10)
B4	B2 (Jump to line 2) AND B5 (Fall-through to line 12)
B5	B6 (Fall-through)
B6	B6 (Jump to line 13) AND Exit (Fall-through)

Insight: Notice how conditional jumps (in B3, B4, and B6) naturally create branching paths, resulting in two outbound edges per block.

Gemini Notebook

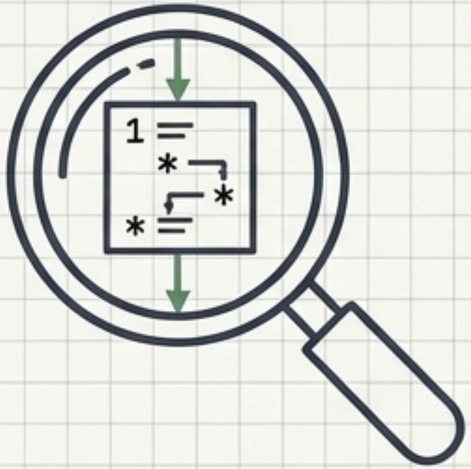
The 17-Line Matrix Identity CFG



Gemini Notebook

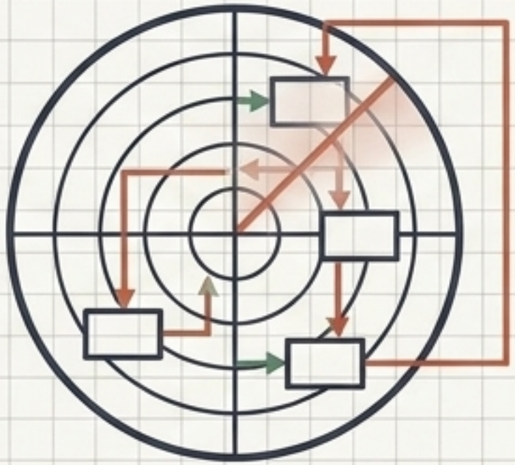
The Power of CFG Abstraction

Local Optimization (Inside the Block)



Because blocks guarantee **straight-line** execution, compilers can safely perform **Dead Code Elimination**, **Common Subexpression Elimination**, and **DAG construction** without worrying about unpredictable jumps.

Global Optimization (Across the Graph)



The CFG structure allows compilers to **map** and analyze **nested loops**, perform **Data Flow Analysis** (liveness, reaching definitions), and optimally **allocate registers** across the entire function.

Synthesis Takeaway: Partitioning unstructured instructions into basic blocks and mapping their Control Flow Graph is the mandatory bridge between raw code translation and highly optimized machine execution.

Gemini Notebook